

PROGRESS REVIEW · JULY 2026

Measuring Data Exposure in LLM-Integrated Productivity Tools

How much of a confidential document do writing-assistant browser extensions silently send to their servers?

Why this matters

40M+

daily Grammarly users

96%

of the Fortune 500 use it

99%

of enterprise users run ≥ 1 browser extension

These tools work by sending the text you type or paste to their own servers for analysis. The processing is server-side — so the content must leave your machine. Most users never see it happen.

And by their own policies: Grammarly keeps your text and, for individual accounts, trains its models on it by default. LanguageTool does neither.

Sources: Grammarly & LanguageTool privacy policies (read July 2026); LayerX 2026.

The question

How does user data exposure differ across LLM-integrated productivity tools during controlled use?

- 1 Define a controlled setup for evaluating these tools
- 2 Measure user data exposure during tool use
- 3 Compare exposure across tools under identical conditions
- 4 Summarise each tool with a clear exposure + confidence assessment

From the project proposal

How the measurement works



The canary is the key idea: a string that exists nowhere else, so finding it in the traffic is undeniable proof the document left the machine.

The tools — mitmproxy & the CA certificate



The CA (certificate authority): mitmproxy makes its own certificate, and we trust it only inside the test profile — so the browser lets us read its own encrypted traffic. Legitimate because it is our machine, our browser, our certificate.

Kali Linux VM

Isolated Firefox profiles

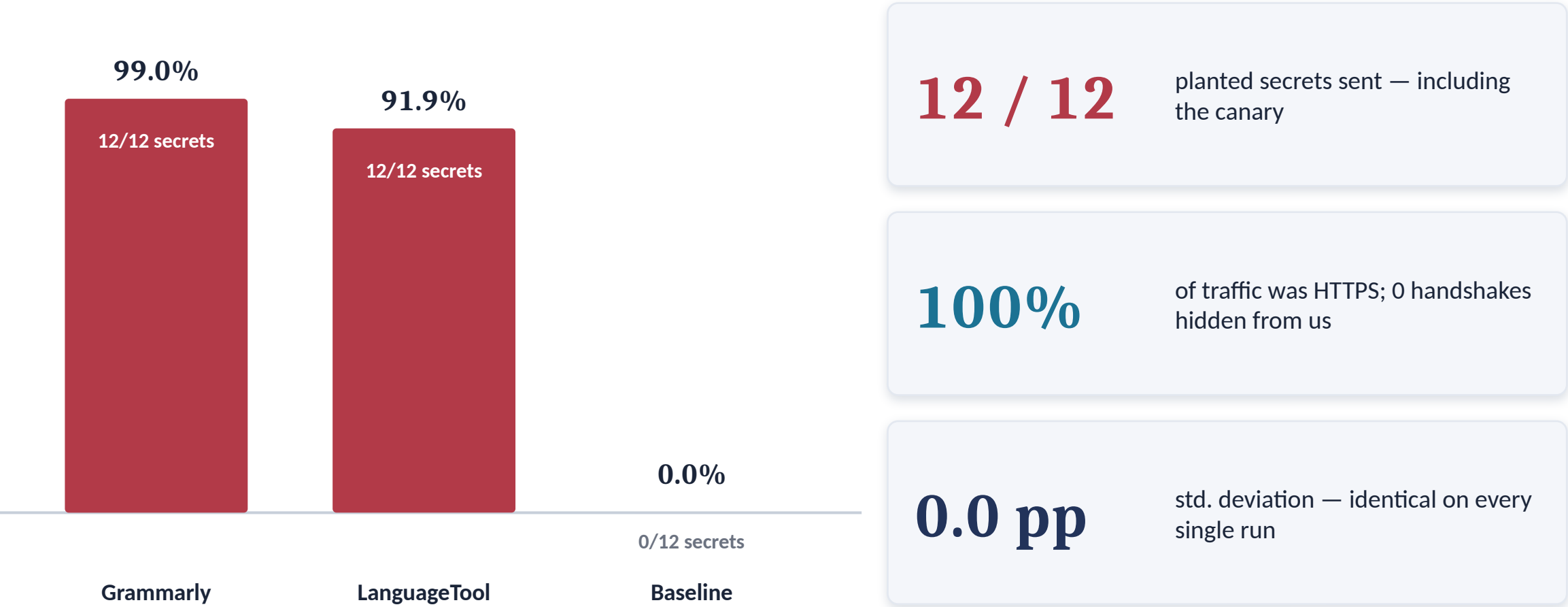
mitmproxy 12.2.3

Python (stdlib) analyzer

A controlled, reproducible setup

- ✓ **Isolated:** Kali Linux VM; a separate Firefox profile per tool, only that tool's extension active
- ✓ **Unambiguous:** a synthetic “confidential memo” (2,065 chars) with 12 planted identifiers + a UUID canary
- ✓ **Repeatable:** 5 runs per tool + 3 baseline runs, an identical locked protocol every time
- ✓ **Honest:** a real manual Ctrl+V (a scripted paste doesn’t trigger the tool); un-decryptable traffic is counted, not hidden

Result: both tools sent the whole document



Both tools sent 100% of the content and all 12 secrets — the 99 vs 92 gap is whitespace only.

The 12 planted secrets — all transmitted

1	Helena Voss name	7	XREF-291-ALPHA document reference
2	HV-2026-391847 employee ID	8	NHK3-RES-7741 reserve fund code
3	Theodora Baumgartner-Klein name	9	AC-2026-00293-DELTA approval code
4	theodora.baumgartner@priv-research-demo.invalid email	10	2026-LGL-00847 contract ID
5	+49 30 4827-9153 phone number	11	DE-291-847-3309 tax registration
6	Project Nighthawk-3 project codename	12	CANARY-BC267061-67DC-485B-8E51-6F5494765CAB UUID canary

12 / 12 transmitted by both Grammarly and LanguageTool — including the canary.

The proof: one unforgeable token

CANARY-BC267061-67DC-485B-8E51-6F5494765CAB

This string exists nowhere else on the internet.

It appeared in the traffic bound for the tool's servers, seconds after an ordinary paste. There is no innocent explanation — it could only have come from the confidential document. This is the headline result: not a percentage, but undeniable, per-secret proof.

Where this sits in the research

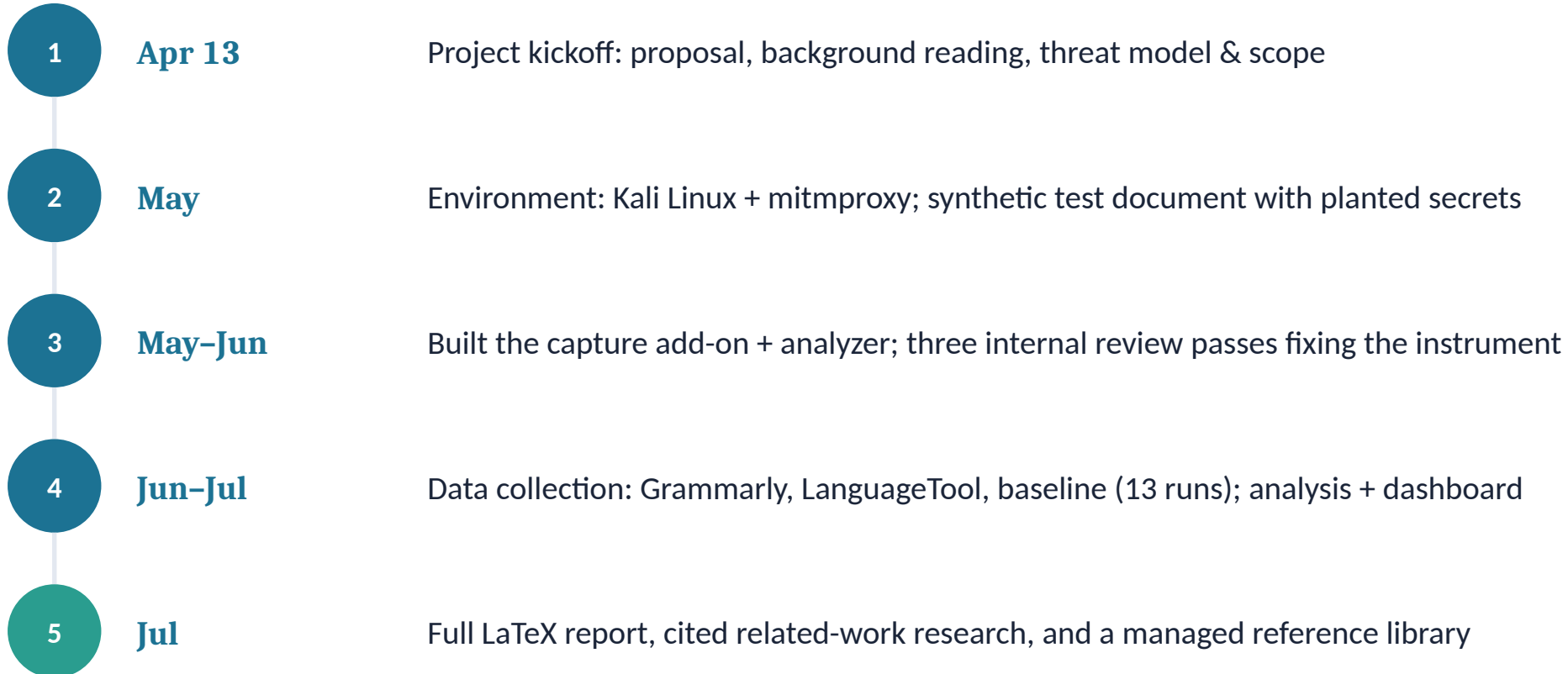
PRIOR WORK

How MANY extensions leak: Starov (WWW'17), Mystique (CCS'18, 181k extensions), Arcanum (USENIX'24 — 3,028 extensions, up to 144M users). **Auditing AI assistants:** Vekaria “Big Help or Big Brother?” (USENIX'25) used the same MITM-decryption method on GenAI assistants.

MY CONTRIBUTION

No one has measured HOW MUCH of a specific confidential document writing assistants transmit, with per-secret canary proof. This is the writing-assistant, canary-proven complement to Vekaria — the same interception principle, narrowed to one reproducible threat model and strengthened so every individual leak is provable, not just observed in aggregate.

Three months of work



Artifacts produced: ~2,000 lines of Python · 13 timestamped capture runs · 15+ documents · a compiled research report · a cited literature digest

Against the proposal: delivered

DELIVERED

- ✓ Controlled setup + identical input
- ✓ Exposure measured (content, bytes, requests, domains)
- ✓ No-extension baseline for comparison
- ✓ Per-tool result + confidence (consistency & completeness)
- ✓ mitmproxy + Python + JSON, as proposed

Verdict: the method fully meets the proposal as specified.

Where I'd value your guidance

?

Is the methodology sound as it stands — anything you'd add or change in how I measured it?

?

Is this the scope you expected — or did you have other tools or directions in mind (e.g. a desktop app or an AI-assistant extension)?

?

Is there a required report template or format — length, structure, citation style — and do you need the standard declaration of originality?

Thank you

Questions & discussion

Marawan Eldeib · marawandeep13@gmail.com · Matrikelnummer 3764796